Slip4

Q.1) Write a C program to find whether a given files passed through command line arguments are present in current directory or not.[10 Marks]

```
#include <stdio.h>
#include <sys/stat.h>
int main(int argc, char *argv[]) {
if (argc < 2) {
printf("Usage: %s <file1> <file2> ...\n", argv[0]);
return 1;
}
struct stat fileStat;
for (int i = 1; i < argc; i++) {
if (stat(argv[i], &fileStat) == 0) {
printf("File '%s' exists in current directory.\n", argv[i]);
} else {
printf("File '%s' does NOT exist in current directory.\n", argv[i]);
}
}
return 0;
}
```

Q.2) Write a C program which creates a child process and child process catches a signal SIGHUP,SIGINT and SIGQUIT. The Parent process send a SIGHUP or SIGINT signal after every 3seconds, at the end of 15 second parent send SIGQUIT signal to child and child terminates by displaying message "My Papa has Killed me!!!".

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

void handle_sighup(int sig)
 {
    printf("Child received SIGHUP\n");
}

void handle_sigint(int sig)
{
    printf("Child received SIGINT\n");
}

void handle_sigquit(int sig)
 {
    printf("Child received SIGQUIT\n");
    exit(0); // Terminate the child process
```

```
}

int main()
 {
    pid_t pid = fork();

    if (pid < 0)
     {
        printf("fork failed");
        exit(0);
    }

    if (pid == 0)

     {
         // Set up signal handlers
        signal(SIGHUP, handle_sighup);
        signal(SIGINT, handle_sigint);
        signal(SIGQUIT, handle_sigquit);

        // Keep the child alive to catch signals
                while (1)
               {
                 pause(); // Wait for signals
                }
        }
         else
            {
               for (int i = 0; i < 5; i++)
             {
                   // Send SIGHUP or SIGINT alternately
                    if (i % 2 == 0)
                    {
                           kill(pid, SIGHUP);
                     }
                     else
                          {
                          kill(pid, SIGINT);
                           }
            sleep(3); // Wait for 3 seconds
        } // Send SIGQUIT after 15 seconds
        kill(pid, SIGQUIT);
        wait(NULL); // Wait for child to terminate
        printf("Parent process terminating.\n");
    }

    return 0;
}
```